

濟南大學

本科毕业设计



基于 Web 的线上代码仓库智能 社交平台的设计与实现

学 院	信息科学与工程学院
专 业	人工智能
班 级	智能 2201
学 生	石政飏
学 号	202131235009
指导教师	张坤

二〇二六年五月

济南大学毕业论文（设计）诚信承诺书

本人声明所呈交的毕业论文（设计）是我个人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得济南大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

学生签名：_____ 日期：_____年____月____日

济南大学毕业论文（设计）使用授权书

本人完全了解济南大学有权保留本人所送交毕业论文（设计）的复印件和电子文档，可以采用影印、缩印或其他复制手段保存论文。本人电子文档的内容和纸质论文的内容相一致。除在保密期内的保密论文外，允许论文被查阅和借阅，可以公布（包括刊登）论文的全部或部分内容。论文的公布（包括刊登）授权济南大学教务处办理。

学生签名：_____ 导师签名：_____

日期：_____年____月____日 日期：_____年____月____日

摘 要

本文以本人完成的 Relax-Git 项目为研究对象,面向毕业设计演示中“仓库能运行、代码能浏览、讨论能落到具体文件和行”的需求,设计了一套基于 Web 的线上代码仓库智能社交平台。系统没有复刻完整代码托管站点,而是把仓库导入、快照生成、文件树浏览、行级评论、通知和私信作为主链路。前端使用 Next.js、React 和 TypeScript 组织社区页、仓库详情页、快照页、消息中心和控制台;后端使用 NestJS、Fastify、Prisma 提供认证、仓库、快照、评论、通知和聊天接口;PostgreSQL 保存业务数据,Redis 与 BullMQ 负责快照任务队列,Go Worker 执行 Git 克隆、worktree 创建和 bundle 生成。本文结合本地启动、接口联调和浏览器页面验证,对需求分析、总体架构、数据库模型、关键模块实现和测试过程进行说明。测试结果表明,Relax-Git 能够完成从发现仓库、浏览代码、围绕文件行讨论到接收通知和私信沟通的连续演示流程。

关键词: 代码仓库; 开发者社交; 快照浏览; 行级评论; 前后端分离

ABSTRACT

With the growth of open-source ecosystems and online collaborative development, code repositories have become more than places for storing source code. They are also important carriers for project presentation, technical discussion, knowledge accumulation and developer connection. To address the problems of scattered demonstration paths and insufficient interaction around repository content in graduation design projects, this thesis designs and implements Relax-Git, a Web-based intelligent social platform for online code repositories. The system takes repositories as the basic content unit and organizes its main workflow around repository import, snapshot browsing, line-level comments and lightweight developer relationships. The front end is implemented with Next.js, React and TypeScript, while the back end is built with NestJS, Fastify and Prisma. PostgreSQL, Redis, BullMQ and a Go Worker are used for data persistence, asynchronous snapshot processing and real-time status notification. The platform supports user authentication, repository management, community feed, file tree browsing, repository comments, line-level discussions, notifications, friend requests, private messages, online status, a lightweight administration console and a repository-oriented intelligent assistant. Through modular design and local demonstration verification, the system forms a complete loop from repository discovery to code discussion, developer communication and platform state observation. This thesis presents the requirement analysis, overall architecture, database design, key module implementation and testing process of the system.

Key words: code repository, developer social platform, snapshot browsing, line-level comment, front-end and back-end separation

目 录

摘 要	I
ABSTRACT	II
1 绪论	1
1.1 选题背景与意义	1
1.2 国内外研究与应用现状	1
1.3 主要研究内容	2
1.4 论文组织结构	2
1.5 本章小结	3
2 相关技术概述	4
2.1 前端开发技术	4
2.2 后端服务与接口技术	4
2.3 数据存储与队列技术	4
2.4 Git 快照与实时通信技术	5
2.5 本章小结	5
3 系统需求分析	6
3.1 系统业务流程	6
3.2 可行性分析	7
3.3 系统角色与用例分析	7
3.4 功能需求分析	9
3.5 非功能需求分析	10
3.6 本章小结	10
4 系统总体设计	12
4.1 总体设计思路	12
4.2 系统总体架构	12
4.3 功能模块设计	13
4.4 数据库结构设计	13
4.5 核心数据表设计	15
4.6 接口与数据流设计	16
4.7 实时通信与异步任务设计	17
4.8 权限与安全设计	17
4.9 本章小结	18
5 系统详细设计与实现	19

5.1 用户认证与身份管理模块	19
5.2 仓库导入与仓库管理模块	19
5.3 快照生成与代码浏览模块	20
5.4 社区互动与行级评论模块	21
5.5 通知、好友与私信模块	21
5.6 智能仓库助手与运营控制台	22
5.7 前端页面实现	23
5.8 工程组织与运行配置	27
5.9 本章小结	28
6 系统测试与结果分析	29
6.1 测试环境	29
6.2 功能测试	29
6.3 接口测试	30
6.4 兼容性与可演示性测试	31
6.5 异常场景测试	31
6.6 测试结果分析	32
6.7 本章小结	32
结 论	33
参考文献	34
附 录	35
致 谢	36

1 绪论

1.1 选题背景与意义

在软件开发活动中，代码仓库承担着版本管理、项目协作、知识沉淀和成果展示等多重作用。随着 Git 工作流、开源社区和社会化编程模式的发展，开发者往往通过仓库主页、提交历史、代码评审和讨论记录理解一个项目的技术路线。GitHub 等平台证明了仓库透明性和开发者互动能够提高协作效率，相关研究也指出社会化编程环境中的可见性、反馈和协作网络会影响项目质量^[5]。因此，将代码仓库作为内容载体构建面向开发者的社交平台，具有明确的现实背景和工程价值。

在本科毕业设计场景中，系统不仅需要能够运行，还需要能够被清楚地讲解、演示和复现。传统代码管理类项目容易停留在仓库增删改查、文件浏览或简单评论层面，缺少从内容发现、代码理解、互动讨论到开发者连接的完整链路。本文选题围绕 Relax-Git 展开，系统将仓库导入、快照浏览、行级评论、通知、好友和私信整合为统一平台，使代码内容本身成为社区互动的中心。该选题能够综合体现 Web 全栈开发、数据库设计、异步任务处理、实时通信和平台化管理等工程能力，也契合人工智能专业对智能平台、人机协作和数据组织方式的训练要求。

本课题的意义主要体现在三个方面。第一，系统以完整软件流程为基础，覆盖需求分析、架构设计、数据库建模、接口开发、前端实现和测试验证，有助于综合运用软件工程知识^[1]。第二，平台围绕代码仓库组织社交关系，避免把社交功能做成与系统主题割裂的独立聊天系统，使内容层、互动层和关系层形成闭环。第三，项目采用本地可稳定运行的演示方式，降低公网部署和外部服务不确定性，便于答辩时展示核心功能、运行状态和关键实现。

1.2 国内外研究与应用现状

国外成熟代码协作平台已经形成较完整的产品形态。GitHub、GitLab 等平台以 Git 仓库为基础，提供代码托管、问题管理、合并请求、代码评审、项目主页和社区互动等功能。Dabbish 等对 GitHub 社会化编程的研究表明，透明的项目活动和开发者可见性能够促进协作与学习^[5]；Bacchelli 和 Bird 对现代代码评审的研究指出，轻量化审查不仅用于发现缺陷，也用于知识共享和团队沟通^[6]；Gousios 等围绕 pull-based 开发模式的研究表明，围绕仓库的贡献流程已经成为开源协作的重要组织方式^[7]。这些研究为本文围绕仓库构建互动平台提供了理论依据。

国内代码托管和开发者社区也在持续发展，相关产品在企业研发管理、开源社区建设和高校实践教学均有应用。但对于本科毕业设计而言，直接复刻大型代码托管

平台并不现实。大型平台往往包含持续集成、项目管理、制品仓库、复杂权限审批等大量能力，系统边界较宽，答辩时不易聚焦。本文将系统定位为面向毕业设计演示的开发者代码社交平台，保留仓库内容、快照浏览、评论互动、私信连接和运行控制台等核心能力，将通用群聊、复杂后台和泛聊天 AI 等能力降级或冻结，从而保证系统主题清晰、功能链路稳定。GitHub issue 模板使用情况的最新研究也说明，模板化协作信息有助于提升问题描述的规范性和后续沟通效率[22]。

结合往年 Web 应用类毕业设计的写法，论文结构通常需要让“需求、设计、实现、测试”之间能互相对应。本文在整理 Relax-Git 时没有把功能截图直接堆到实现章节，而是先在需求分析中确定游客、普通用户、仓库成员和管理员的边界，再在总体设计中把 Repository、BaseSnapshot、Comment、Notification、Chat 等实体与仓库详情页、快照页、消息中心和控制台对应起来。这样写的原因是本项目的重点不只是登录和增删改查，而是仓库快照、文件行评论、通知推送与私信沟通之间的链路是否能连续演示。近年来关于代码评审研究方法、数据集和指标的系统映射也表明，围绕需求、实现与验证建立清晰对应关系，是评价协作型开发工具的重要基础[19]。

1.3 主要研究内容

本文的主要研究内容包括以下几个方面：一是分析开发者代码社交平台的角色、业务场景和功能需求，明确系统以仓库为内容单位、以快照和行级评论为互动核心；二是设计前后端分离、多服务协同的总体架构，确定 Web 前端、API 服务、Go Worker、PostgreSQL 和 Redis 的职责边界；三是建立用户、仓库、分支、快照、评论、通知、好友和私信等核心实体的数据模型；四是实现仓库导入、分支识别、异步快照生成、文件树浏览和代码内容读取等关键功能；五是实现社区 feed、点赞收藏、仓库评论、行级评论、通知反馈、好友申请和私信沟通；六是设计面向答辩演示的轻量控制台和仓库发现助手，提升系统可讲解性和完整性。

1.4 论文组织结构

全文共分为六章。第一章介绍选题背景、研究现状、研究内容和论文结构。第二章介绍系统开发涉及的前端、后端、数据存储、队列、Git 快照和实时通信等相关技术。第三章先说明业务流程与可行性，再从系统角色、用例规约、功能需求和非功能需求角度分析系统建设目标。第四章说明系统总体设计，包括总体架构、功能模块、数据库结构、接口数据流和安全设计。第五章结合实际代码实现，阐述用户认证、仓库管理、快照浏览、社区互动、通知私信、智能助手、控制台和前端页面等模块。第六章给出测试环境、测试用例、接口测试和结果分析。最后对全文工作进行总结，并展望后续优化方向。

1.5 本章小结

本章完成了选题背景、研究现状和论文组织说明。对 **Relax-Git** 来说，选题的落点不是做一个大而全的 **GitHub** 替代品，而是把毕业设计中可演示、可解释、可复现的链路收窄到仓库内容社交：用户能够导入仓库，后台生成快照，前端展示文件树和代码内容，评论可以挂到文件或具体行，后续通知和私信把讨论接上。后续章节将围绕这条链路展开技术选型、需求建模、架构设计、模块实现和测试验证。

2 相关技术概述

2.1 前端开发技术

Relax-Git 前端采用 Next.js、React、TypeScript、Tailwind CSS、React Query 和 Zustand 等技术。Next.js 基于 React 提供应用路由、页面组织和构建能力, 适合构建多页面 Web 应用^[10]。React 采用组件化思想, 将页面拆分为可复用组件, 使仓库卡片、评论面板、通知铃铛、消息中心和快照代码浏览器等界面元素能够独立维护^[11]。TypeScript 在 JavaScript 基础上增加静态类型约束, 能够减少接口字段、组件属性和状态管理过程中的低级错误。

样式层采用 Tailwind CSS, 以原子化类名组织布局、间距、颜色和响应式规则, 适合快速实现统一风格的页面。数据请求层采用 React Query, 负责接口请求、缓存、加载状态和错误状态管理, 使社区列表、仓库详情、通知列表和消息列表等数据能够在页面之间稳定刷新。全局轻量状态采用 Zustand, 用于维护登录态、消息状态、通知状态和应用级状态。前端还使用 Socket.IO Client 接收后端实时事件, 使快照状态、评论、通知和私信能够及时反馈到界面。

2.2 后端服务与接口技术

后端采用 NestJS 与 Fastify 构建 API 服务。NestJS 借鉴模块化和依赖注入思想, 将系统划分为 auth、users、repositories、community、snapshots、comments、notifications、chats、admin、ai 和 health 等业务模块, 每个模块通过 controller 暴露接口, 通过 service 组织业务逻辑^[12]。这种结构有利于将认证、仓库、快照、评论、消息和控制台聚合等功能分离, 降低不同模块之间的耦合。

接口设计以 RESTAPI 为主, 配合 Swagger 文档展示接口路径、参数和返回结构。REST 架构强调资源、统一接口和无状态交互, 适合本系统中用户、仓库、快照、评论、通知和私信等资源型对象^[8]。认证方面采用 JWT 令牌, 用户登录后由后端签发 token, 前端在后续请求中携带令牌访问受保护接口。JWT 本身是一种紧凑的声明传输格式, 适用于前后端分离场景下的身份认证^[18]。

2.3 数据存储与队列技术

系统核心业务数据存储在后端 PostgreSQL 中, 并通过 Prisma 进行数据访问。PostgreSQL 具备成熟的事务、约束、索引和关系建模能力, 适合保存用户、仓库、分支、快照、评论、通知、好友关系和私信消息等结构化数据^[14]。Prisma 通过 schema 文件统一描述模型、字段和关系, 并生成类型安全的数据访问客户端, 有助于减少手

写 SQL 带来的字段错误和关联错误^[13]。

Redis 在系统中承担缓存、队列和发布订阅等职责^[15]。仓库导入和快照生成属于耗时任务，如果在普通 HTTP 请求中同步完成，用户需要长时间等待，且接口容易受网络 and 仓库大小影响。因此系统将快照任务写入队列，由后台 Worker 异步消费。BullMQ 基于 Redis 提供任务队列能力，能够支持任务入队、消费、失败处理和状态追踪。API 服务与 Worker 之间通过数据库状态和 Redis 消息协同，使前台页面能够感知快照处理进度。

2.4 Git 快照与实时通信技术

Git 是分布式版本控制系统，能够通过提交对象、分支和引用记录项目历史^[4]。Relax-Git 并不试图完整实现代码托管平台，而是利用 Git 仓库 URL、分支信息、commitSha、worktree 和 bundle 等能力，将远程仓库在本地转换为可浏览的快照。基础快照保存某一仓库分支在指定提交上的代码状态，会话快照则面向具体用户提供临时浏览上下文。通过这种设计，系统可以把代码文件、路径和行号作为评论锚点，实现围绕具体代码行的讨论。

实时通信采用 Socket.IO 实现，其底层思想与 WebSocket 协议的全双工通信能力一致^[17]。系统在用户连接后将其加入用户房间、仓库房间、聊天房间或快照房间，并根据业务事件推送通知、私信、评论和快照状态。对于代码社交平台而言，实时通信的价值不在于构建复杂聊天系统，而在于让用户能及时看到仓库讨论、快照处理和私信互动的变化，从而提升平台反馈能力。

2.5 本章小结

本章介绍了 Relax-Git 使用的主要技术。前端技术负责页面组织、组件化开发、状态管理和实时事件展示；后端技术负责模块化接口、身份认证和业务逻辑；PostgreSQL、Prisma、Redis 和 BullMQ 负责数据持久化、类型安全访问和异步任务；Git 快照与 WebSocket 实时通信支撑了系统区别于普通社区平台的核心能力。这些技术共同构成了后续系统设计和实现的基础。

3 系统需求分析

3.1 系统业务流程

Relax-Git 的核心业务流程围绕“仓库内容社交”展开。用户首先完成注册和登录，然后可以在社区浏览已经发布的公开仓库，也可以导入自己的 Git 仓库。仓库导入后，系统读取远程分支信息，创建仓库记录和基础快照任务，后台 Worker 处理 Git 克隆、worktree 创建和 bundle 生成。快照就绪后，用户进入仓库详情或快照页面，浏览文件树和代码内容，并围绕仓库、文件或具体代码行发表评论。评论、回复、点赞和解决状态会触发通知，使相关用户能够继续参与讨论。若用户希望进一步交流，可以发送好友申请并创建私聊会话。管理员则通过控制台查看平台统计、最近仓库、最近评论和系统状态。

从业务闭环看，系统不是将仓库管理、社区评论和消息中心简单并列，而是让它们沿着同一条主线协作。仓库导入解决内容来源问题，快照解决代码内容可浏览问题，行级评论解决围绕代码讨论的问题，通知和私信解决讨论后的持续连接问题，控制台则解决系统运行状态可观察问题。该流程能够在答辩演示中以一条连续路径呈现，避免模块之间相互割裂。

仓库导入与快照生成流程

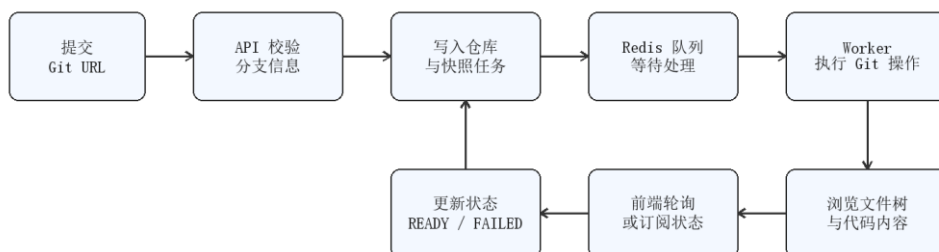


图 3.1 仓库导入与快照生成流程

表 3.1 系统核心业务流程

阶段	参与对象	主要操作	结果
身份进入	普通用户	注册、登录、同步用户态	获得访问受保护功能的身份凭证
仓库准备	普通用户	导入 Git URL、验证分支、维护仓库信息	形成可管理的仓库记录
快照处理	API、Redis、Worker	创建快照任务、消费队列、生成 worktree 和 bundle	快照状态变为 READY 或 FAILED
内容互动	普通用户	浏览社区、进入仓库、查看文件树、发表仓库或行级评论	围绕代码内容形成讨论记录

阶段	参与对象	主要操作	结果
关系延续	普通用户	接收通知、添加好友、发送私信	公开互动延伸为开发者连接
运行观察	管理员	访问控制台、查看平台数据和服务状态	掌握系统运行与内容动态

3.2 可行性分析

技术可行性方面，系统采用的 Next.js、React、NestJS、Prisma、PostgreSQL、Redis、Socket.IO 和 Go 均为成熟技术，具有较完善的文档和社区资料。前端使用组件化方式组织页面，后端使用模块化方式组织业务，数据库模型由 Prisma Schema 统一维护，后台 Git 操作交给 Go Worker 异步执行。上述技术能够支撑本课题提出的仓库导入、快照浏览、行级评论、私信和控制台等功能。

经济可行性方面，项目主要依赖开源框架和本地开发环境，不需要购买商业软件或专用服务器。PostgreSQL 与 Redis 可以通过 Docker Compose 在本地启动，Web 与 API 可以通过 npm 启动，Worker 可以通过 Go 命令运行。该方式满足毕业设计演示需求，也便于在答辩前进行多次彩排和故障排查。

操作可行性方面，系统界面流程接近常见 Web 平台。普通用户通过导航进入社区、仓库、消息和个人中心，管理员通过控制台查看运行状态。仓库导入、快照浏览和评论操作均在页面中通过表单、按钮和列表完成，用户不需要直接操作数据库或命令行。对于开发者而言，项目根目录 README 已给出本地启动路径，降低了复现难度。

3.3 系统角色与用例分析

系统主要包含游客、普通用户、仓库所有者、仓库成员和管理员五类角色。游客可以浏览部分公开社区内容，但不能进行评论、收藏、私信等需要身份的操作。普通用户是系统主要使用者，可以维护个人资料、浏览社区、导入仓库、发表评论、发送好友申请和私信。仓库所有者可以编辑仓库信息、发布仓库、管理成员和处理加入申请。仓库成员可以访问被授权的仓库内容并参与讨论。管理员用于演示平台化管理能力，可以查看控制台中的概览数据、最近仓库、热门仓库、最近评论和基础服务状态。

表 3.2 系统角色与主要用例

角色	主要用例	权限边界
游客	浏览公开社区、查看公开仓库概要	不能评论、收藏、私信或访问私有仓库
普通用户	登录、浏览仓库、点赞收藏、评论、好友申请、私信	只能访问公开仓库和被授权仓库
仓库所有者	导入仓库、编辑仓库、发布仓库、管理成员	拥有本人仓库的管理权限
仓库成员	查看授权仓库、浏览快照、参与评论	按成员关系访问内部或私有仓库

角色	主要用例	权限边界
管理员	查看控制台、观察平台统计和服务状态	具备管理端访问权限

表 3.3 典型用例描述

用例名称	前置条件	基本流程	后置结果
导入仓库	用户已登录，Git URL 可访问	填写仓库地址和描述，后端验证 URL，读取分支并创建快照任务	仓库记录保存，快照进入队列
浏览快照	仓库可访问，快照状态就绪	进入快照页面，展开文件树，选择文件并请求文件内容	页面展示代码内容并支持行级交互
发表行级评论	用户已登录并可访问快照	选择代码行，输入评论内容并提交	评论与文件路径、行号关联，其他用户可回复
接收通知	用户与评论或回复存在关联	评论服务生成通知，WebSocket 推送或通知列表拉取	未读数量更新，用户可跳转到对应内容
发送私信	双方为好友或可建立私聊	创建私聊会话，发送文本消息，服务端推送到会话房间	消息保存并更新未读状态
查看控制台	管理员已登录	请求控制台聚合接口，前端展示统计和状态	管理员获得平台运行概览

从用例关系看，普通用户的操作围绕仓库内容展开，仓库所有者是普通用户在特定仓库上的扩展角色，管理员则提供平台观察视角。系统没有将群聊、聊天室和泛聊天 AI 作为主用例，避免功能边界偏离代码仓库社交这一核心主题。

Relax-Git 系统用例图

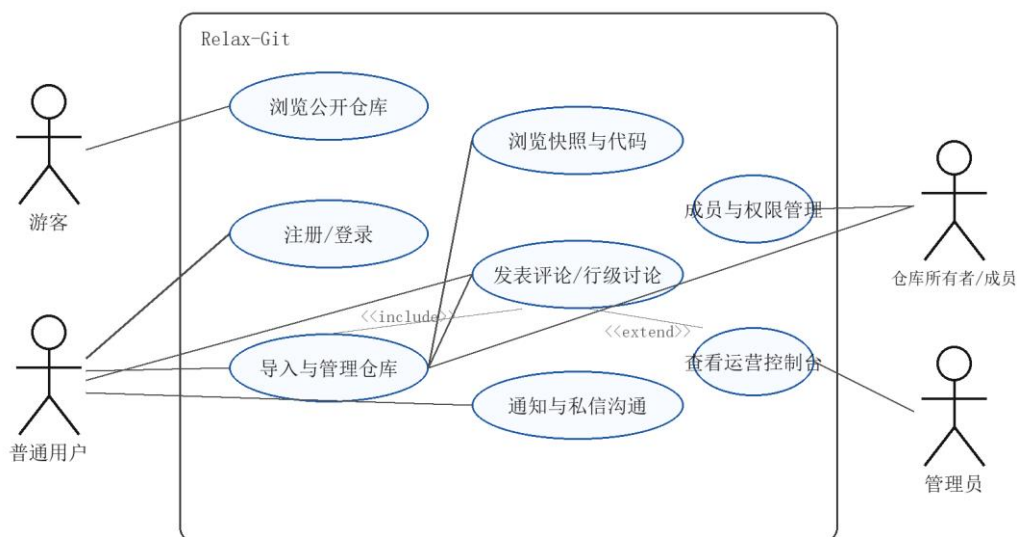


图 3.2 Relax-Git 系统用例图

为使角色边界能够落到后续设计和测试中，本文在用例图基础上补充核心用例规

约。规约重点描述参与者、前置条件、基本流程和执行后的结果，用于约束第 4 章模块设计、第 5 章实现说明以及第 6 章测试用例。

表 3.4 核心用例规约

编号	用例名称	参与者	前置条件	基本流程与后置结果
UC01	用户注册登录	普通用户	用户未登录，已打开登录或注册页面	提交用户名、UID 和密码；后端校验并签发 JWT；前端保存登录态。异常时返回错误提示。
UC02	导入仓库并生成快照	普通用户、仓库所有者	用户已登录，Git 地址可访问	填写仓库信息并提交；后端校验 URL、读取分支、创建仓库和快照任务；Worker 生成 worktree 与 bundle。
UC03	浏览快照并行级评论	普通用户、仓库成员	仓库可访问，快照状态为 READY	进入仓库或快照页；展开文件树并读取文件；选择代码行发表评论；系统记录文件路径和行号。
UC04	处理通知并发送私信	普通用户	用户已登录，存在评论回复或好友关系	查看通知并标记已读；搜索用户、发送好友申请；建立私聊后发送消息并更新未读数。
UC05	查看运营控制台	管理员	管理员已登录	访问控制台入口；后端校验角色；聚合用户、仓库、评论、快照和服务状态并返回页面展示。

3.4 功能需求分析

表 3.5 系统主要功能需求

功能模块	主要功能	实现目标
用户身份体系	注册、登录、退出、个人资料、角色识别	保证用户态控制和普通用户、管理员入口区分
仓库内容体系	仓库导入、仓库管理、分支获取、封面上传、发布到社区	以 Git 仓库为平台内容单位
快照浏览体系	基础快照、会话快照、文件树、文件内容、快照状态	支持稳定浏览仓库代码并承载行级评论
社区互动体系	社区 feed、浏览量、点赞、收藏、仓库评论、评论点赞	形成公开内容分发与互动反馈
代码讨论体系	快照评论、文件级评论、行级评论、回复、解决评论	围绕代码具体位置展开讨论
关系与消息体系	好友申请、好友列表、私信、在线状态、未读统计	补充开发者之间的轻量连接
运行管理体系	控制台概览、仓库动态、最近评论、系统状态	展示平台运行和管理能力
智能辅助体系	热门仓库推荐、方向检索、仓库简介生成	辅助仓库发现与理解

功能关系上，仓库内容体系和快照浏览体系是平台基础，社区互动体系和代码讨论体系是平台特色，关系与消息体系使互动能够从公开讨论延伸到开发者连接，运行管理体系用于体现平台化管理视角，智能辅助体系作为加分项服务于仓库发现和仓库理解。

为保证第 3 章提出的需求能够在后续章节中得到设计、实现和测试支撑，本文建立功能需求追踪矩阵。矩阵中的每一项均对应第 4 章的模块或数据设计、第 5 章的实

现模块以及第 6 章的测试用例，避免需求、设计、实现和测试之间出现脱节。

表 3.6 需求-设计-实现-测试追踪矩阵

需求项	设计对应	实现对应	测试对应
用户注册登录与身份管理	4.3 功能模块设计、4.8 权限与安全设计	5.1 用户认证与身份管理模块	6.2 TC01、6.3 认证接口测试
仓库导入与仓库管理	4.3 功能模块设计、4.6 接口与数据流设计	5.2 仓库导入与仓库管理模块	6.2 TC02、6.3 仓库接口测试
快照生成与代码浏览	4.4 数据库结构设计、4.7 异步任务设计	5.3 快照生成与代码浏览模块	6.2 TC04、6.3 快照接口测试、6.4 主链路检查
社区互动与行级评论	4.3 功能模块设计、4.8 权限与安全设计	5.4 社区互动与行级评论模块	6.2 TC03、TC05、6.3 评论接口测试
通知、好友与私信沟通	4.7 实时通信设计、4.5 消息相关表	5.5 通知、好友与私信模块	6.2 TC06、TC07、TC08、6.3 通知与私信接口测试
智能仓库助手与运营控制台	4.3 模块设计、4.6 控制台接口设计	5.6 智能仓库助手与运营控制台	6.2 TC09、6.4 控制台可演示性测试
权限与安全控制	4.8 权限与安全设计	5.1、5.2、5.3 中的守卫和路径校验	6.2 TC10、6.5 EC01-EC07 异常场景测试

3.5 非功能需求分析

表 3.7 系统非功能需求

需求类别	具体要求
可用性	系统应能在本地稳定运行 Web、API、Worker、PostgreSQL 和 Redis，核心演示链路可连续完成。
安全性	用户密码采用哈希存储，接口通过 JWT 鉴权，私有仓库和控制台需进行权限校验。
可维护性	前端按页面和组件组织，后端按业务模块组织，数据库访问由 Prisma 统一封装。
可扩展性	仓库快照任务从 API 请求中剥离，由 Redis 队列和 Go Worker 异步处理，便于后续扩展任务类型。
一致性	仓库、快照、评论和通知之间应保持关联一致，删除仓库或快照时通过外键级联处理相关数据。
可演示性	系统需提供清晰的本地启动说明、健康检查和控制台状态，使答辩过程可控。

除上述需求外，系统还需要关注异常可观测性。由于 Git 仓库大小、远程访问速度和本地文件系统状态会影响快照任务，系统需要在数据库中记录 QUEUED、PROCESSING、READY 和 FAILED 等状态，并在发生错误时保存错误信息，便于前端展示和答辩调试。

3.6 本章小结

本章先从业务流程和可行性角度确认 Relax-Git 的建设基础，再围绕角色用例、功能需求和非功能需求展开分析。系统以仓库导入和快照浏览为内容基础，以评论、

通知、好友和私信为互动延伸，以控制台和智能助手体现平台特色。需求追踪矩阵进一步明确需求与后续设计、实现、测试之间的对应关系，为第四章的总体设计提供依据。

4 系统总体设计

4.1 总体设计思路

系统总体设计遵循“内容中心、链路闭环、模块解耦、演示稳定”的原则。内容中心是指平台围绕代码仓库组织数据和页面，仓库详情、分支、快照、文件树、代码行和评论都从仓库派生；链路闭环是指用户从发现仓库到代码讨论，再到通知和私信连接，能够形成完整使用路径；模块解耦是指前端展示、后端接口、异步任务、数据库和缓存队列各自承担独立职责；演示稳定是指系统优先保证本地环境可运行、主链路可展示，而不追求过重的公网部署或复杂后台。

4.2 系统总体架构

Relax-Git 采用前后端分离和多服务协同架构。Web 前端负责页面渲染、用户交互、请求缓存和实时消息展示；API 服务负责认证、仓库、社区、快照、评论、通知、私信和控制台等业务接口；PostgreSQL 保存用户、仓库、快照和互动数据；Redis 用于队列、缓存和状态发布；Go Worker 负责执行耗时的 Git 仓库处理任务。该架构将高频交互请求和耗时快照处理分离，避免用户导入仓库时长时间阻塞 API 请求。

Relax-Git 系统总体架构

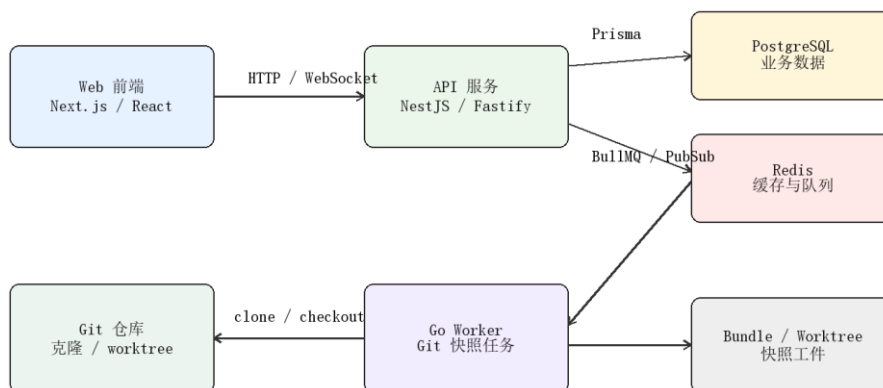


图 4.1 Relax-Git 系统总体架构

从运行链路看，浏览器只直接访问 Web 前端和 API 服务，数据库与缓存不暴露给用户。API 服务通过 Prisma 访问 PostgreSQL，通过 Redis 与 BullMQ 创建或订阅任务状态。Worker 只关注仓库快照任务，与用户会话、社区 feed 和私信等高频业务解耦。这样的架构既能表现多服务协同能力，又能在本地环境中保持较低运维复杂度。

4.3 功能模块设计

根据需求分析结果，系统功能模块划分为用户认证、仓库管理、快照浏览、社区互动、评论讨论、通知中心、好友私信、智能助手、运营控制台和健康检查等部分。各模块在后端以 NestJS module 组织，在前端以路由页面、业务组件和状态 store 组织。模块划分既对应用户可见功能，也对应后端代码目录，便于开发、测试和论文阐述保持一致。

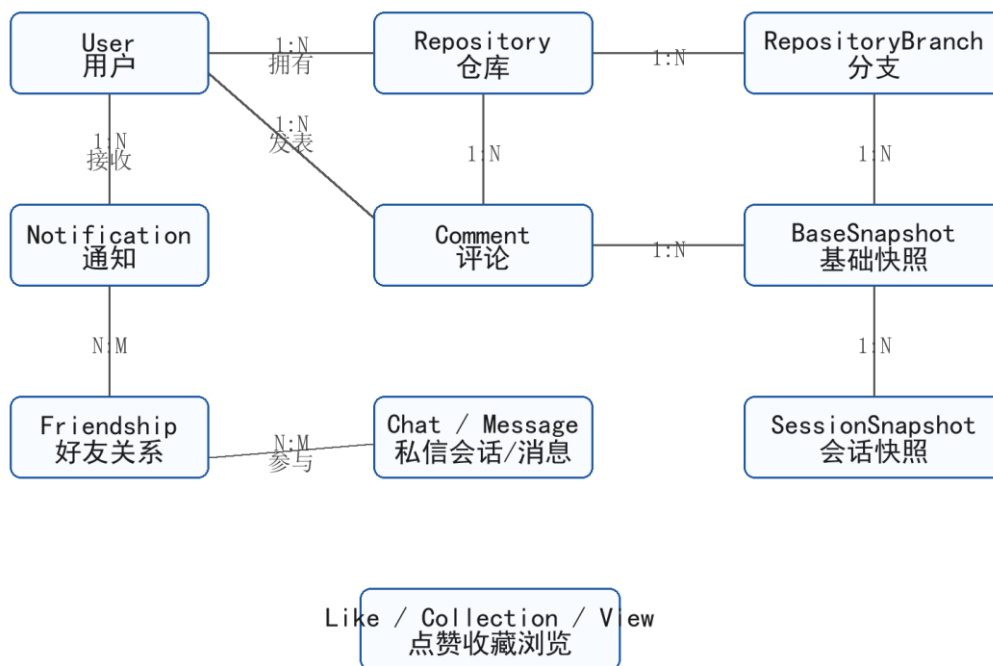
表 4.1 系统功能模块设计

模块	前端表现	后端支撑	核心数据
用户认证	登录注册页、用户菜单、个人中心	auth、users、JWT 守卫	User、UserSession
仓库管理	仓库导入表单、仓库详情、设置弹窗	repositories、members、join-requests	Repository、RepositoryBranch、Member
快照浏览	快照页面、文件树、代码查看器	snapshots、artifacts、worker	BaseSnapshot、SessionSnapshot、SnapshotArtifact
社区互动	社区 feed、仓库卡片、仓库评论	community、repository-interaction	RepositoryLike、RepositoryCollection、RepositoryView
代码评论	行级评论面板、评论列表、解决状态	comments、websocket	Comment、CommentLike、Notification
好友私信	消息中心、好友列表、好友申请抽屉	chats、chat/friends、online-status	Friendship、FriendRequest、Chat、Message
运营控制台	控制台概览、系统状态、最近动态	api/admin/console、health	聚合统计与健康检查结果
智能助手	仓库发现助手与 AI 状态	ai、search	仓库摘要、搜索历史和推荐结果

4.4 数据库结构设计

数据库采用 PostgreSQL，并通过 Prisma Schema 统一维护模型定义。核心实体包括 User、Repository、RepositoryBranch、BaseSnapshot、SessionSnapshot、SnapshotArtifact、Comment、Notification、Friendship、FriendRequest、Chat、Message、RepositoryLike、RepositoryCollection 和 RepositoryView 等。用户与仓库之间通过所有者、成员和加入申请建立关系；仓库与分支、基础快照和快照工件关联；快照与评论、时间线和搜索历史关联；用户之间通过好友关系、好友申请、私信会话和消息可见性形成轻量社交结构。

Relax-Git 核心 E-R 图



说明：图中列出论文主体涉及的核心实体，RepositoryLike、RepositoryCollection、RepositoryView 归并为仓库互动统计。

图 4.2 Relax-Git 核心 E-R 图

E-R 图中，User 是用户身份和关系的起点，Repository 是平台内容组织的核心。RepositoryBranch、BaseSnapshot 和 SessionSnapshot 共同描述仓库代码状态，Comment、Notification、Chat 和 Message 则承接围绕代码内容产生的讨论与消息。点赞、收藏和浏览记录作为仓库互动统计关系存在，用于社区展示和控制台统计。

表 4.2 主要数据实体说明

实体名称	含义	关键字段或关系
User	平台用户	username、uid、password、role、isOnline
Repository	代码仓库	gitUrl、ownerId、visibility、isPublished、stars、viewCount
RepositoryBranch	仓库分支	repoId、name、commitSha、isDefault
BaseSnapshot	基础快照	repoId、branchId、commitSha、status、worktreePath
SessionSnapshot	会话快照	baseSnapshotId、userId、expiresAt、status
Comment	锚点评论	snapshotId、authorId、anchorType、filePath、lineStart、lineEnd
Notification	通知	userId、actorId、type、commentId、isRead

实体名称	含义	关键字段或关系
Friendship / Chat	好友与私信	好友关系、私信会话、消息与已读状态

4.5 核心数据表设计

用户与仓库相关表用于保存平台的身份基础和内容入口。其中 `users` 表记录用户角色、登录状态和在线时间，`repositories` 表记录仓库所有者、可见性、发布状态和社区统计字段，成员与申请表则用于表达仓库协作边界。

表 4.3 用户与仓库相关表结构

数据表	关键字段	设计说明
<code>users</code>	<code>id</code> 、 <code>username</code> 、 <code>uid</code> 、 <code>password</code> 、 <code>role</code> 、 <code>isOnline</code> 、 <code>lastSeenAt</code>	保存平台用户、角色和在线状态， <code>password</code> 保存哈希结果
<code>user_sessions</code>	<code>userId</code> 、 <code>token</code> 、 <code>expiresAt</code>	保存用户会话令牌，用于学习项目中的 JWT 管理
<code>repositories</code>	<code>name</code> 、 <code>gitUrl</code> 、 <code>ownerId</code> 、 <code>visibility</code> 、 <code>isPublished</code> 、 <code>tags</code> 、 <code>language</code>	保存仓库基础信息、社区发布状态和展示字段
<code>repository_branches</code>	<code>repoId</code> 、 <code>name</code> 、 <code>commitSha</code> 、 <code>isDefault</code>	记录仓库分支和对应提交，用于创建基础快照
<code>members</code>	<code>repoId</code> 、 <code>userId</code> 、 <code>role</code>	记录仓库成员关系和成员角色
<code>join_requests</code>	<code>repoId</code> 、 <code>userId</code> 、 <code>status</code> 、 <code>reviewedAt</code>	记录用户申请加入仓库的流程状态

快照与代码讨论相关表用于支撑“仓库导入后能够浏览代码并围绕具体文件讨论”的核心链路。基础快照保存分支级代码状态，会话快照保存用户临时浏览状态，评论表通过文件路径和行号把讨论定位到具体代码上下文。

表 4.4 快照与代码讨论相关表结构

数据表	关键字段	设计说明
<code>base_snapshots</code>	<code>repoId</code> 、 <code>branchId</code> 、 <code>commitSha</code> 、 <code>status</code> 、 <code>worktreePath</code> 、 <code>bundlePath</code>	保存分支级基础快照和 Worker 处理状态
<code>session_snapshots</code>	<code>baseSnapshotId</code> 、 <code>userId</code> 、 <code>status</code> 、 <code>expiresAt</code> 、 <code>lastAccessedAt</code>	保存用户临时浏览会话，支持过期和续期
<code>snapshot_artifacts</code>	<code>repoId</code> 、 <code>commitSha</code> 、 <code>status</code> 、 <code>worktreePath</code> 、 <code>bundlePath</code>	按 <code>commit</code> 复用快照工件，避免重复处理同一提交
<code>comments</code>	<code>snapshotId</code> 、 <code>authorId</code> 、 <code>anchorType</code> 、 <code>filePath</code> 、 <code>lineStart</code> 、 <code>lineEnd</code>	保存快照、文件和代码行评论
<code>timeline_events</code>	<code>repoId</code> 、 <code>type</code> 、 <code>actorId</code> 、 <code>payload</code>	记录仓库创建、快照就绪和评论等事件
<code>search_history</code>	<code>userId</code> 、 <code>repositoryId</code> 、 <code>snapshotId</code> 、 <code>query</code> 、 <code>searchType</code>	保存仓库搜索和快照搜索历史

社区与消息相关表用于延续仓库内容产生的互动。点赞、收藏和浏览记录用于社

区排序和热度统计，通知表用于承接评论回复、系统事件和提及，聊天与消息表用于支持开发者之间的后续私信沟通。

表 4.5 社区与消息相关表结构

数据表	关键字段	设计说明
repository_likes	repoId、userId、createdAt	记录用户对仓库的点赞关系，并通过唯一索引避免重复点赞
repository_collections	repoId、userId、createdAt	记录用户收藏仓库的关系
repository_views	repoId、userId、ipAddress、viewedAt	记录登录或匿名用户的仓库浏览行为
notifications	userId、actorId、type、commentId、repoId、isRead	保存评论回复、提及和系统类通知
friendships	userId、friendId、createdAt	保存双向好友关系中的单向记录
chats、messages	chatId、senderId、content、isRead、readAt	保存私聊会话、消息内容和已读状态

数据库索引主要围绕查询入口设计。例如仓库按 ownerId、visibility 查询，快照按 repoId、branchId、status 查询，评论按 snapshotId、authorId、filePath 查询，通知按 userId 和 isRead 查询，消息按 chatId、createdAt 和 isRead 查询。外键级联删除用于保证仓库、快照、评论等强关联数据的一致性，避免删除仓库后遗留不可访问的评论或快照记录。

4.6 接口与数据流设计

系统接口以 REST API 为主，并辅以 WebSocket 事件推送。仓库导入、快照、评论、社区、通知、私信和控制台接口分别支撑内容创建、代码浏览、互动反馈和运行管理；表 4.6 列出主要接口入口，表 4.7 说明关键数据流。

表 4.6 主要接口设计

模块	方法	路径
认证注册	POST	/auth/register
认证登录	POST	/auth/login
仓库导入	POST	/api/repositories/import
仓库详情	GET	/api/repositories/:id
快照树	GET	/api/snapshots/:id/tree
快照文件	GET	/api/snapshots/:id/file
社区列表	GET	/api/community/feed
浏览记录	POST	/api/community/repositories/:id/view
评论创建	POST	/api/comments
评论查询	GET	/api/comments/by-snapshot/:snapshotId
通知列表	GET	/api/notifications
未读数量	GET	/api/notifications/unread-count
创建私聊	POST	/api/chats/direct
发送私信	POST	/api/chats/:chatId/messages
控制台	GET	/api/admin/console/overview

表 4.7 关键数据流说明

数据流	输入	处理	输出
登录鉴权	用户名、密码	认证服务校验密码并签发 JWT	用户信息、访问令牌
仓库导入	Git URL、仓库名称、可见性	验证 URL、读取分支、写入仓库、创建快照任务	仓库 ID、分支信息、快照状态
快照浏览	snapshotId、path	校验权限、定位 worktree、读取目录或文件	文件树节点或文件内容
评论互动	snapshotId、锚点、评论内容	校验权限、写入评论、生成通知、推送事件	评论详情和实时事件
私信通信	chatId、消息内容	校验成员关系、写入消息、推送到聊天房间	消息记录和未读数量
控制台概览	管理员身份	聚合用户、仓库、评论、快照和服务健康状态	控制台概览数据

4.7 实时通信与异步任务设计

实时通信主要由 WebSocket Gateway 负责。用户建立连接后，服务端根据认证信息识别用户身份，并将连接加入 `user:{userId}` 房间；用户进入聊天页面时加入 `chat:{chatId}` 房间；进入快照页面时加入 `snapshot:{snapshotId}` 房间。这样可以将通知、私信、评论和快照状态精确推送到目标用户或目标页面，避免所有用户接收无关事件。

表 4.8 WebSocket 事件设计

事件名称	触发场景	接收对象
join:chat	用户进入私聊会话	当前用户连接
join:snapshot	用户进入快照页面	当前用户连接
snapshot:status-changed	Worker 更新快照状态	订阅该快照的用户
comment:new	用户创建评论或回复	订阅该快照的用户
comment:updated	评论内容或状态变化	订阅该快照的用户
notification:new	评论回复、提及或系统事件	目标用户房间
message:new	用户发送私信	聊天会话房间
unread-counts	通知或消息未读数变化	目标用户房间

异步任务主要用于仓库快照生成。API 服务负责接收用户请求和创建任务，Redis 队列负责保存待处理任务，Go Worker 负责消费队列并执行 Git 操作，PostgreSQL 保存任务最终状态。任务状态从 QUEUED 进入 PROCESSING，处理成功后变为 READY，处理失败则变为 FAILED 并记录错误信息。前端通过轮询接口或 WebSocket 状态事件展示处理进度。

4.8 权限与安全设计

系统通过 JWT 进行身份认证,通过用户角色和仓库访问守卫进行权限控制。普通业务接口默认需要登录,少量社区浏览接口允许匿名访问但会在用户登录时返回个性化互动状态。仓库权限由所有者、成员关系、仓库可见性和管理员角色共同决定,私有仓库仅允许所有者、成员和管理员访问。控制台接口通过角色守卫限制为管理员访问。密码采用 bcrypt 哈希存储,环境变量中保存数据库、Redis 和 AI 服务配置,本地演示不提交真实密钥。

文件访问安全是快照模块的重点。用户读取文件树和文件内容时,后端需要基于快照的 `worktreePath` 解析相对路径,并限制路径不能逃逸到工作树之外,避免通过构造路径访问系统其他文件。同时,快照访问需要校验仓库可见性和成员关系,防止用户绕过仓库详情页直接请求私有快照内容。类似 npm 生态中安全相关代码评审的实证研究表明,路径、权限和依赖边界是 Web 工程安全审查中的高频关注点[20]。

表 4.9 权限控制策略

资源类型	允许访问者	控制方式
公开仓库	游客、登录用户、管理员	根据 <code>visibility</code> 和发布状态判断
私有仓库	所有者、成员、管理员	仓库访问守卫校验 <code>ownerId</code> 和成员关系
快照内容	具备仓库访问权限的用户	快照接口先校验仓库权限再读取文件
评论操作	登录且可访问快照的用户	评论服务校验用户身份和锚点合法性
私信会话	会话成员	聊天接口校验 <code>chatId</code> 与用户成员关系
控制台	管理员	角色守卫校验 <code>UserRole.ADMIN</code>

4.9 本章小结

本章完成了 Relax-Git 的总体设计。系统采用 Web 前端、NestJS API、Go Worker、PostgreSQL 和 Redis 协同的架构,并围绕仓库内容社交划分功能模块。数据库设计描述了核心实体及关系,接口和数据流设计说明了主要资源路径和业务流转方式,实时通信与异步任务设计保证了快照处理和互动反馈,权限与安全设计则保证了用户身份、仓库访问和文件读取的基本安全性。

5 系统详细设计与实现

5.1 用户认证与身份管理模块

用户认证模块由 AuthModule、UsersModule 和全局 JWT 守卫共同完成。用户注册时提交用户名、UID 和密码，后端对密码进行哈希处理后写入 users 表；用户登录成功后获取访问令牌，前端在后续请求中携带令牌访问受保护接口。后端在 AppModule 中注册全局 JwtAuthGuard，使大部分业务接口默认处于认证保护之下，只有通过 Public 装饰器标记的接口才允许匿名访问。

表 5.1 认证模块关键实现

实现点	涉及位置	说明
注册登录接口	auth.controller、auth.service	提供 register、login、logout、profile、verify 等接口
密码处理	AuthService	注册时哈希密码，登录时校验哈希结果
全局鉴权	AppModule、JwtAuthGuard	默认保护业务接口，公共接口通过装饰器放行
用户状态	auth-store、auth-sync	前端保存用户态并在页面间同步
管理员入口	session role、console page	根据 role 展示控制台入口并访问管理接口

身份信息不仅用于接口鉴权，还会影响页面入口和业务行为。普通用户可以访问社区、仓库、快照、评论、个人中心和消息中心；管理员用户登录后可以在导航中看到控制台入口，并访问 /api/admin/console/overview 聚合接口。WebSocket 连接同样通过 JWT 完成身份校验，连接建立后服务端将用户加入 user:{userId} 房间，用于后续通知、私信和未读消息推送。

5.2 仓库导入与仓库管理模块

仓库管理模块围绕 repositories 控制器和 service 实现。用户可以创建仓库、查询仓库列表、查看仓库详情、更新仓库信息、删除仓库、验证 Git 地址、获取分支列表和上传仓库封面。系统提供 /api/repositories/import 接口用于导入 Git 仓库，该接口完成 URL 验证、仓库记录创建或复用、分支信息获取和快照任务创建，并返回 repositoryId、branch 信息和快照相关标识。

表 5.2 仓库导入处理步骤

步骤	处理内容	异常处理
URL 接收	读取用户提交的 gitUrl、仓库名称和可见性	字段为空时返回参数错误
地址校验	匹配 HTTPS、SSH、GitHub、GitLab 等地址形式	地址格式不合法时拒绝导入

步骤	处理内容	异常处理
远程探测	通过 <code>git ls-remote</code> 获取远程分支和默认分支	远程不可达时返回验证失败
仓库写入	创建或复用 <code>Repository</code> 和 <code>RepositoryBranch</code>	重复仓库按 <code>ownerId</code> 与 <code>name</code> 唯一约束处理
任务创建	生成 <code>BaseSnapshot</code> 并写入 <code>Redis</code> 队列	队列失败时保留仓库记录并返回错误提示
结果返回	返回仓库 <code>ID</code> 、分支信息和快照状态	前端进入状态展示或仓库详情页

为了使仓库既能作为个人资产又能作为社区内容，`Repository` 模型中包含 `visibility`、`isPublished`、`publishedAt`、`coverImage`、`tags`、`language`、`stars`、`viewCount` 和 `trendingScore` 等字段。用户可以将仓库发布到社区，社区 `feed` 根据发布时间、热度或浏览量进行排序。仓库可见性分为公开、内部和私有，后端在读取仓库、分支和快照时通过 `RepoAccessGuard` 进行权限判断，避免用户访问无权限的私有仓库。

5.3 快照生成与代码浏览模块

快照模块是系统区别于普通社区平台的关键部分。系统将快照分为基础快照和会话快照。基础快照面向仓库分支，是可复用的代码状态；会话快照面向具体用户，用于临时代码浏览和交互。`BaseSnapshot` 保存 `repoId`、`branchId`、`commitSha`、`status`、`worktreePath`、`bundlePath` 和访问统计等字段；`SessionSnapshot` 保存 `baseSnapshotId`、`userId`、`expiresAt` 和 `status`，用于控制临时浏览会话的生命周期。

表 5.3 快照状态流转

状态	含义	前端展示
QUEUED	任务已写入队列，等待 <code>Worker</code> 消费	显示排队中或准备中
PROCESSING	<code>Worker</code> 正在克隆仓库、创建 <code>worktree</code> 或生成 <code>bundle</code>	显示处理中并允许刷新状态
READY	快照工件已生成， <code>worktreePath</code> 可读取	展示文件树和代码内容
FAILED	任务处理失败， <code>errorMessage</code> 保存错误原因	显示失败原因并提供重试或重新导入提示
EXPIRED	会话快照超过 <code>TTL</code>	提示重新创建或续期会话

当用户导入仓库或创建快照时，API 将任务写入 `Redis` 队列，`Go Worker` 从队列消费任务并执行 `Git` 操作。`Worker` 的处理流程包括更新状态为 `PROCESSING`、克隆仓库、检查仓库大小、创建指定提交的 `worktree`、生成 `bundle`、写回 `worktreePath` 和 `bundlePath`、更新状态为 `READY`；如果发生错误，则记录 `errorMessage` 并将状态更新为 `FAILED`。API 端通过统一快照控制器提供 `/api/snapshots/:id`、`/tree`、`/file`、`/extend` 和释放接口，前端据此展示文件树和代码内容。

代码浏览模块在读取文件时需要同时处理展示效果和安全边界。后端根据 `snapshotId` 找到对应 `worktreePath`，再将用户传入的相对路径解析为绝对路径，并检查该路径仍位于 `worktree` 根目录内。目录读取时跳过隐藏文件和 `.git` 目录，文件读

取时返回文本内容、文件路径和语言提示，前端再结合代码高亮组件展示行号和行级评论入口。

快照状态变化通过 Redis 发布订阅和 WebSocket 推送给前端。WebSocket Gateway 订阅 snapshot:status:* 频道，解析 Worker 发布的状态消息后向 snapshot:{snapshotId} 房间广播 snapshot:status-changed 事件。这样前端可以在快照处理过程中及时展示状态，而不需要持续阻塞等待。

5.4 社区互动与行级评论模块

社区模块提供公开仓库 feed、仓库详情、浏览记录、热门标签、热门语言、仓库评论和评论点赞等功能。/api/community/feed 支持游标分页、排序、语言过滤、标签过滤和关键词搜索，返回仓库名称、描述、封面、标签、语言、点赞数、浏览量、评论数、作者信息以及当前用户是否点赞或收藏。浏览记录通过 RepositoryView 和 RepositoryViewsAgg 记录，为热度计算和控制台展示提供数据基础。

表 5.4 社区互动数据写入

互动类型	数据表	业务说明
浏览	repository_views、 repository_views_agg	记录用户或匿名访问，并按天聚合浏览量
点赞	repository_likes、 repositories.stars	记录用户点赞关系并维护仓库点赞数
收藏	repository_collections	记录用户收藏关系，用于个人收藏列表
仓库评论	comments 或社区评论接口	围绕仓库整体进行讨论
行级评论	comments	记录 filePath、lineStart、lineEnd 等锚点字段
评论点赞	comment_likes	记录用户对评论的反馈

评论模块支持项目级、快照级、文件级和行级锚点。Comment 模型中的 anchorType 表示评论锚点类型，filePath、lineStart 和 lineEnd 用于定位到具体文件和代码行。用户在快照页面选择代码行后，可以提交行级评论；其他用户可以回复、点赞或将评论标记为已解决。评论创建、更新、解决和删除后，WebSocket Gateway 会向对应快照房间推送 comment:new、comment:updated、comment:resolved 或 comment:deleted 事件，从而保证多人浏览同一快照时能够收到互动反馈。

5.5 通知、好友与私信模块

通知模块用于承接评论回复、提及和系统事件。Notification 模型包含 userId、actorId、type、commentId、snapshotId、repoId、content、isRead 和 createdAt 等字段。用户可以获取通知列表、未读数量、单条标记已读、批量标记已读或全部标记已读。通知不仅在个人中心展示，也可以通过 WebSocket 的 user 房间推送给目标用户。

表 5.5 好友与私信处理流程

功能	处理流程	结果
搜索用户	根据关键词查询用户列表并过滤自身	返回可申请好友的用户信息
发送申请	写入 FriendRequest, 状态为 PENDING	对方可在申请列表中处理
接受申请	更新申请状态并写入 Friendship 关系	双方可在好友列表中看到对方
创建私聊	根据两个用户 ID 生成 directKey 并复用已有会话	返回 DIRECT 类型会话
发送消息	写入 Message, 推送 message:new 事件	接收方看到新消息和未读数量
标记已读	更新 lastReadAt 或消息已读字段	未读计数减少

好友和私信模块构成平台的轻量关系层。用户可以搜索其他用户、发送好友申请、接受或拒绝申请、查看好友列表和删除好友。私信会话以 DIRECT 类型为主，系统冻结群聊创建和成员邀请能力，避免项目偏离代码社交主线。用户创建私聊后可以获取消息列表、发送消息、清空会话和标记已读，服务端通过 chat:{chatId} 房间推送新消息和已读回执，并通过 unread-counts 接口和事件反馈未读数量。

5.6 智能仓库助手与运营控制台

智能辅助模块不作为泛聊天系统，而是定位为仓库发现助手。AI 控制器提供服务状态检查、仓库助手问答、提交信息生成和代码审查接口，其中仓库助手支持 discovery、thinking 和 chat 三种模式。当前毕业设计主线中重点保留热门仓库推荐、按技术方向查找仓库和仓库简介生成，用于增强社区内容发现和仓库理解能力。机器学习应用于源代码的综述研究显示，代码理解、摘要和审查建议是智能化开发工具中较常见且边界清晰的应用方向[21]。

表 5.6 控制台概览数据

数据类别	展示内容	数据来源
平台统计	用户总数、仓库总数、已发布仓库数、评论总数、快照总数	Prisma 聚合查询
最近仓库	仓库名称、所有者、可见性、发布时间	Repository 与 User 关联查询
热门仓库	点赞数、浏览量、热度分数	Repository 社区字段和浏览聚合
最近评论	评论内容摘要、作者、仓库名称、创建时间	Comment、User、Repository 关联查询
服务状态	API、Worker、Database、Redis	健康检查和连接测试

运营控制台用于展示开发者对平台运行状态的管理能力。管理员访问 /api/admin/console/overview 后，可以获取用户总数、仓库总数、已发布仓库数、评论总数、快照总数、最近仓库、热门仓库、最近评论以及 API、Worker、Database、Redis

等基础状态。控制台不追求复杂 RBAC 或通用 CMS，而是服务于答辩演示和日常观察，使系统形成用户端、管理端、服务端和后台任务的完整闭环。

5.7 前端页面实现

前端页面位于 `apps/web/src/app` 和 `apps/web/src/components` 目录，采用路由页面、业务组件和基础 UI 组件分层组织。页面层负责获取路由参数和组合业务组件，业务组件负责仓库卡片、社区过滤器、评论面板、快照代码查看器、消息中心和好友申请抽屉等具体功能，基础 UI 组件负责按钮、输入框、对话框、标签页、弹窗和提示等通用交互。这样的组织方式使页面功能与组件复用保持平衡。

表 5.7 前端主要页面与组件

页面或组件	位置	实现内容
社区页	<code>app/community</code> 、 <code>components/community</code>	展示公开仓库 feed、筛选条件、仓库卡片和仓库详情入口
仓库页	<code>app/repositories</code> 、 <code>components/repository</code>	导入仓库、仓库详情、成员管理、加入申请和仓库讨论
快照页	<code>app/snapshots/[id]</code> 、 <code>components/snapshot</code>	文件树、代码内容、行级评论和快照状态展示
消息中心	<code>components/messages</code>	好友列表、好友申请、私聊会话和未读消息状态
通知入口	<code>components/notifications-bell</code> 、 <code>notification-row</code>	展示未读通知、通知列表和跳转链接
控制台	<code>app/console/page.tsx</code>	展示平台概览、最近仓库、热门仓库、最近评论和服务状态
状态管理	<code>stores/auth-store</code> 、 <code>messages-store</code> 、 <code>notifications-store</code>	维护登录态、消息状态和通知状态

在交互实现上，社区列表、仓库详情、通知和消息列表通过请求缓存减少重复加载；快照代码浏览器根据文件路径请求文件内容，并通过行号定位评论锚点；Socket Provider 统一建立实时连接，使通知、私信和快照状态能够在不同页面间共享。前端没有单独构建复杂后台应用，而是在同一 Web 应用中为管理员提供控制台入口，符合轻量演示平台的定位。

为了验证页面实现效果，本文在本地演示环境中启动 PostgreSQL、Redis、API 服务和 Web 前端，并使用浏览器自动化工具截取系统真实运行界面。图 5.1 至图 5.5 展示了社区发现、仓库详情、快照讨论、代码浏览和接口文档等关键页面，能够对应前文提出的“社区发现仓库—进入仓库详情—浏览代码快照—围绕快照讨论—查看接口能力”的主链路。



图 5.1 社区仓库发现界面

社区页将公开仓库作为内容卡片展示，并提供搜索、排序和标签信息。用户可以从仓库名称、描述、语言、标签、浏览量和点赞数等信息初步判断仓库价值，再进入仓库详情页继续查看代码和讨论内容。



图 5.2 仓库详情概览界面

仓库详情页通过概览、分支、讨论和成员四个标签组织仓库信息。概览页展示默认分支、可见性和创建时间等基础字段，分支页进一步承接快照代码浏览，讨论页承接围绕仓库和快照的评论互动。



图 5.3 仓库快照讨论界面

讨论页中的评论数据来自后端 `comments` 模块,评论与快照关联后可以保留在具体代码状态之上。该设计避免评论只停留在普通留言板层面,使讨论能够和仓库版本、文件内容以及代码浏览页面形成联系。

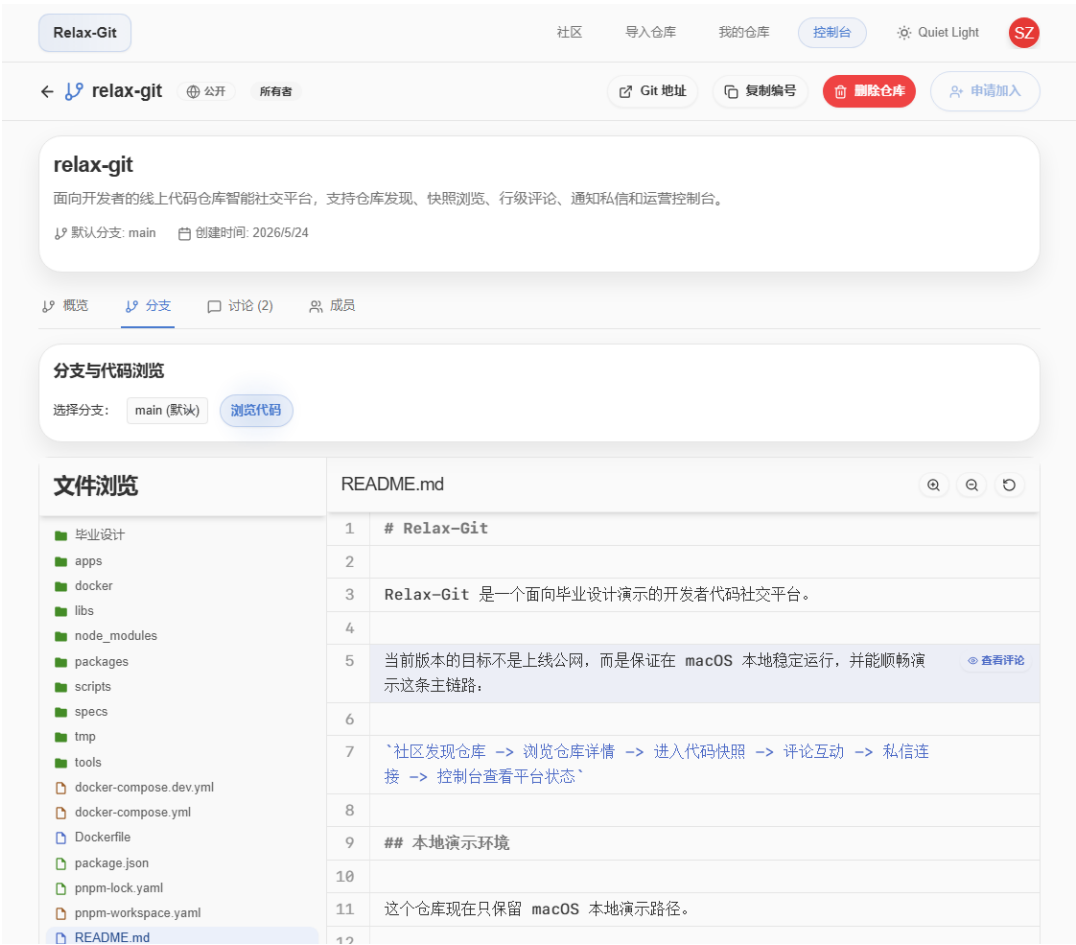


图 5.4 分支与代码快照浏览界面

分支与代码浏览页面左侧展示工作树文件结构，右侧展示选中文件的代码内容，并在代码行上保留评论入口。前端根据仓库、分支、会话快照和文件路径向后端请求文件树与文件内容，使用户能够在浏览代码时直接定位讨论上下文。

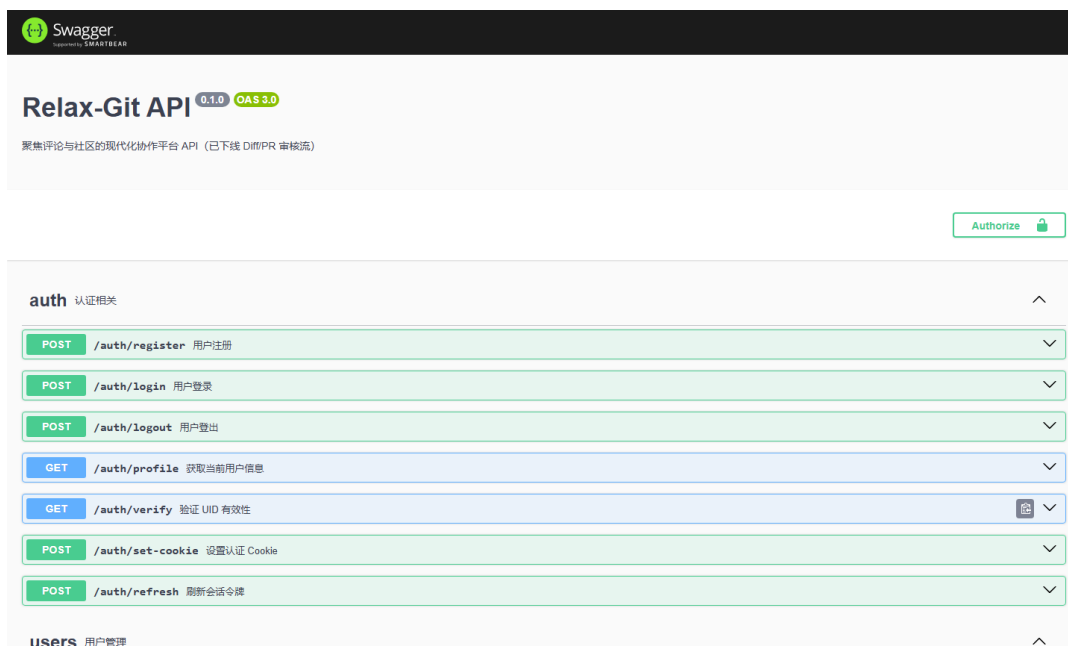


图 5.5 后端 Swagger 接口文档界面

Swagger 文档展示了后端认证、用户、仓库、快照、评论、通知、私信和控制台等接口分组。该页面用于本地联调和接口验收，也体现了后端服务对前端页面的支撑关系。

5.8 工程组织与运行配置

项目采用多应用目录组织。apps/web 保存 Next.js 前端，apps/api 保存 NestJS API，apps/worker 保存 Go Worker，libs/shared 保存共享类型与 Prisma 生成产物，docker 目录保存数据库初始化与容器配置，specs/graduation-streamline-plan 保存毕业设计收口过程中的设计说明。该结构使前端、后端、后台任务和共享类型边界明确，便于分别构建和调试。

表 5.8 本地运行与工程命令

命令	作用	说明
<code>pnpm install</code>	安装依赖	安装前端、后端和共享库依赖
<code>pnpm db:generate</code>	生成 Prisma Client	根据 schema 生成类型安全客户端
<code>pnpm -C libs/shared build</code>	构建共享库	生成共享类型与工具函数
<code>pnpm docker:dev</code>	启动数据库和缓存	启动 PostgreSQL 与 Redis
<code>pnpm db:setup</code>	初始化数据库	执行数据库准备脚本
<code>pnpm dev</code>	启动 Web 与 API	进入前后端联调状态
<code>cd apps/worker && go run .</code>	启动 Worker	消费仓库快照任务

运行配置主要通过环境变量和示例配置文件控制。数据库连接、Redis 地址、JWT 密钥、Worker 端口和 AI 服务配置均不写死在代码中。本地演示时只需要保证 `.env` 与 Worker 配置一致，数据库和 Redis 正常启动，前后端与 Worker 便可以通过固定端口进行协同。这样的配置方式有利于保持源码整洁，也方便后续迁移到其他环境。

5.9 本章小结

本章结合 Relax-Git 的实际实现说明了各关键模块。用户认证保证接口访问安全，仓库模块提供内容入口，快照模块支撑代码浏览，评论和社区模块形成互动，通知、好友和私信延续用户关系，智能助手和控制台增强平台特色，前端页面和工程配置则将这些能力组织为可操作、可复现、可演示的完整系统。

6 系统测试与结果分析

6.1 测试环境

系统测试以本地演示环境为基础。前端运行在 Node.js 20 及以上环境，使用 pnpm 管理依赖并启动 Next.js 开发服务；后端运行 NestJS API 服务，提供 Swagger 文档、认证接口和业务接口；数据库采用 PostgreSQL，缓存和队列采用 Redis；后台任务服务采用 Go Worker，负责消费快照任务并执行 Git 操作。测试时需要同时启动 Web、API、Worker、PostgreSQL 和 Redis，访问入口包括 Web 首页、API 健康检查、Worker 健康检查和管理员控制台。

表 6.1 测试环境配置

项目	配置或版本要求
运行平台	本地开发环境
前端环境	Node.js 20+、pnpm 8+、Next.js、React
后端环境	NestJS、Fastify、Prisma、Swagger
数据库	PostgreSQL 16 或兼容版本
缓存队列	Redis 6+
后台任务	Go 1.22+、Git 2.0+
辅助工具	Docker Compose、浏览器、API 文档页面

6.2 功能测试

功能测试采用黑盒测试方式，重点验证用户从社区发现仓库、进入仓库详情、查看快照、围绕代码行评论、接收通知、添加好友、发送私信和查看控制台的主链路。测试用例覆盖正常路径和典型异常路径，例如未登录访问受保护页面、普通用户访问私有仓库、普通用户访问控制台等。

表 6.2 核心功能测试用例

编号	测试内容	操作步骤	预期结果
TC01	用户登录	输入合法账号和密码并提交登录表单	登录成功，页面进入用户态并可访问受保护页面
TC02	仓库导入	提交有效 Git URL，填写仓库信息并导入	系统创建仓库记录，生成分支信息并创建快照任务
TC03	社区浏览	进入社区页，按最新或热门排序浏览仓库	页面返回公开仓库列表并展示作者、标签、点赞和浏览量
TC04	快照浏览	进入仓库详情后打开快照页面并选择文件	系统展示文件树和代码内容，快照状态为就绪后可浏览
TC05	行级评论	在代码文件中选择指定行并发表评论	评论保存成功，评论列表显示文件路径和代码行范围

编号	测试内容	操作步骤	预期结果
TC06	通知提醒	用户收到评论回复或相关互动	未读通知数量增加，通知列表可查看并标记已读
TC07	好友申请	搜索用户并发送好友申请，对方接受	双方好友列表更新，能够创建私聊会话
TC08	私信沟通	在消息中心发送文本消息并标记已读	消息发送成功，未读数量和已读状态更新
TC09	控制台查看	管理员进入控制台页面	展示平台概览、最近仓库、最近评论和系统状态
TC10	权限限制	普通用户访问他人私有仓库或控制台	系统拒绝访问并返回权限错误

表 6.3 页面功能测试用例

页面	测试项	预期表现
社区页	筛选、排序、仓库卡片展示	仓库列表按条件刷新，卡片信息完整
仓库详情页	仓库基础信息、成员和讨论入口	能展示仓库元数据并进入相关操作
快照页	文件树展开、文件内容加载、行号显示	文件选择后代码内容正确展示
消息中心	好友列表、好友申请、私聊消息	申请状态和消息列表更新正常
通知入口	未读数量、通知列表、跳转链接	未读数准确，点击后能定位相关内容
控制台	统计卡片、最近动态、服务状态	管理员可查看聚合数据和健康状态

6.3 接口测试

接口测试主要验证接口路径、请求参数、鉴权状态、返回结构和异常处理。认证接口需要验证注册、登录、退出、验证 token 和刷新 token；仓库接口需要验证 URL 校验、仓库导入、仓库详情、分支列表和封面上传；快照接口需要验证快照详情、文件树、文件内容和会话续期；评论接口需要验证按快照查询、创建、点赞、更新、删除和解决评论；控制台接口需要验证管理员角色限制。

表 6.4 接口测试要点

模块	测试接口	关注点
认证	/auth/login、/auth/profile、/auth/verify	token 签发、用户信息返回、未登录拦截
仓库	/api/repositories/import、/api/repositories/:id	Git URL 校验、仓库权限、分支信息
快照	/api/snapshots/:id/tree、/api/snapshots/:id/file	快照状态、路径安全、文件内容返回
评论	/api/comments、/api/comments/by-snapshot/:snapshotId	锚点字段、回复关系、权限校验
通知	/api/notifications、/api/notifications/unread-count	未读数量、标记已读、通知排序

模块	测试接口	关注点
私信	/api/chats/direct、 /api/chats/:chatId/messages	会话创建、消息发送、已读状态
控制台	/api/admin/console/overview	管理员角色、统计聚合、服务状态

6.4 兼容性与可演示性测试

兼容性测试主要关注系统在本地演示环境中的可启动性和浏览器访问效果。系统依赖 Web、API、Worker、PostgreSQL 和 Redis 五个部分，任何一个部分异常都会影响主链路。因此测试前需要依次确认数据库和缓存容器已启动、Prisma Client 已生成、共享类型已构建、Web 与 API 服务已启动、Worker 健康检查可访问。浏览器侧重点检查社区页面、仓库详情页、快照页、消息中心和控制台在常见窗口尺寸下是否能正常展示。

表 6.5 本地演示检查项

检查项	检查方式	通过标准
PostgreSQL 与 Redis	执行 <code>pnpm docker:dev</code> 并查看容器状态	数据库和缓存连接正常
API 服务	访问健康检查或 Swagger 文档	接口服务可访问并返回正常状态
Worker 服务	访问 Worker 健康检查	后台任务服务处于运行状态
Web 前端	访问 <code>http://localhost:3000</code>	页面加载正常，登录态可同步
主链路	按社区、仓库、快照、评论、私信、控制台顺序演示	链路连续，无阻断性错误

6.5 异常场景测试

表 6.6 异常场景测试用例

编号	异常场景	输入或操作	预期处理
EC01	未登录访问受保护接口	不携带 token 请求仓库管理接口	返回未认证错误，前端引导登录
EC02	访问无权限私有仓库	普通用户请求他人私有仓库详情	返回权限错误，不泄露仓库内容
EC03	无效 Git URL	导入格式错误或不可访问的 URL	校验失败并展示错误原因
EC04	快照任务失败	Worker 处理 Git 操作失败	状态更新为 FAILED，记录 errorMessage
EC05	路径越界访问	请求包含 ../ 的文件路径	后端拒绝读取并返回错误
EC06	非管理员访问控制台	普通用户请求 /api/admin/console/overview	返回权限错误
EC07	WebSocket 未认证连接	无 token 建立实时连接	连接被拒绝或无法加入业务房间

异常测试的目的不是追求复杂的攻击验证，而是保证毕业设计演示过程中常见错误能够被明确处理。对于仓库导入失败、快照失败和权限不足等情况，系统需要给出

可理解的状态或错误信息，使用户能够继续操作或重新发起任务，同时也便于开发者在控制台和日志中定位问题。

6.6 测试结果分析

通过对核心链路进行联调，系统能够覆盖用户认证、仓库导入、社区展示、快照浏览、评论互动、通知反馈、好友私信和控制台查看等主要功能。仓库导入与快照生成采用异步处理后，API 请求不需要等待完整 Git 操作完成，前端可以通过状态轮询或 WebSocket 事件感知快照进度。行级评论能够记录文件路径和行号范围，适合围绕具体代码展开讨论。通知和私信功能使公开互动后的开发者连接得到延续。控制台能够从管理视角展示平台数据和服务状态，便于答辩时说明系统完整性。

测试过程中需要重点关注三类问题。第一，Git 仓库大小和网络情况会影响快照任务耗时，因此系统通过仓库大小检查、状态字段和错误信息提高可观测性。第二，快照、评论和仓库之间存在较强关联，接口需要严格校验仓库访问权限，避免未授权用户通过快照或评论接口间接访问私有仓库。第三，私信和通知属于实时交互能力，WebSocket 连接需要在认证失败、断开重连和多端在线时保持状态一致。

从工程工作量角度看，测试覆盖了前端页面、后端接口、数据库关联、Redis 队列、Worker 任务和 WebSocket 推送等多个层面。虽然当前系统以本地演示环境为主，尚未进行大规模并发压测，但测试结果能够证明系统主链路具备完整性，关键异常场景具备基本防护和可解释反馈。

6.7 本章小结

本章围绕本地演示环境、功能测试、接口测试、兼容性测试、异常场景测试和结果分析说明了系统测试情况。测试结果表明，Relax-Git 的主要功能链路和演示链路具备完整性，但在公网部署、智能推荐深度、控制台管理粒度和快照工件清理策略方面仍有继续完善空间。

结 论

本文围绕“基于 Web 的线上代码仓库智能社交平台的设计与实现”完成了 Relax-Git 系统的需求分析、总体设计、详细实现和测试分析。系统以代码仓库为核心内容单位，通过仓库导入、分支识别、基础快照、会话快照、文件树和代码内容浏览构建内容底座；通过仓库评论、行级评论、回复、点赞、解决评论和通知反馈构建互动层；通过好友申请、私信、在线状态和未读消息构建轻量关系层；通过仓库发现助手和运营控制台增强系统特色和管理视角。

从工程实现角度看，系统采用 Next.js、React 和 TypeScript 构建前端页面，采用 NestJS、Fastify 和 Prisma 构建后端服务，采用 PostgreSQL 保存业务数据，采用 Redis 和 BullMQ 支撑队列任务，采用 Go Worker 处理 Git 快照任务，并通过 Socket.IO 推送状态和互动事件。多服务协同设计使系统能够将耗时的 Git 操作从普通 API 请求中剥离，提高用户交互稳定性。

从最终实现情况看，Relax-Git 已能在本地环境按预期跑通主要演示链路：启动 PostgreSQL、Redis、API、Web 和 Go Worker 后，用户可注册登录、导入仓库、等待快照任务完成、进入文件树浏览代码，并在文件或代码行上发表评论；评论、通知、好友和私信模块能够把一次仓库浏览延伸为开发者之间的后续沟通。项目仍有明显可继续完善的地方，例如公网部署和任务监控尚未做成生产级方案，智能仓库助手也主要保留在发现与摘要场景。后续可在现有数据模型基础上补充推荐规则、权限审计和 Worker 任务面板，使系统从答辩演示版本逐步走向更完整的代码社区产品。

参 考 文 献

- [1] 张海藩. 软件工程导论[M]. 北京: 清华大学出版社, 2013.
- [2] 王珊, 萨师煊. 数据库系统概论[M]. 北京: 高等教育出版社, 2014.
- [3] Sommerville I. Software Engineering[M]. 10th ed. Boston: Pearson, 2015.
- [4] Chacon S, Straub B. Pro Git[M]. 2nd ed. New York: Apress, 2014.
- [5] Dabbish L, Stuart C, Tsay J, et al. Social coding in GitHub: transparency and collaboration in an open software repository[C]//Proceedings of CSCW. 2012: 1277-1286.
- [6] Bacchelli A, Bird C. Expectations, outcomes, and challenges of modern code review[C]//Proceedings of ICSE. 2013: 712-721.
- [7] Gousios G, Pinzger M, Deursen A V. An exploratory study of the pull-based software development model[C]//Proceedings of ICSE. 2014: 345-355.
- [8] Fielding R T. Architectural Styles and the Design of Network-based Software Architectures[D]. Irvine: University of California, Irvine, 2000.
- [9] Hohpe G, Woolf B. Enterprise Integration Patterns[M]. Boston: Addison-Wesley, 2003.
- [10] Vercel. Next.js Documentation[EB/OL]. [2026-05-24]. <https://nextjs.org/docs>.
- [11] Meta Open Source. React Documentation[EB/OL]. [2026-05-24]. <https://react.dev/reference/react>.
- [12] NestJS. NestJS Documentation[EB/OL]. [2026-05-24]. <https://docs.nestjs.com>.
- [13] Prisma. Prisma Documentation[EB/OL]. [2026-05-24]. <https://www.prisma.io/docs>.
- [14] PostgreSQL Global Development Group. PostgreSQL Documentation[EB/OL]. [2026-05-24]. <https://www.postgresql.org/docs>.
- [15] Redis Ltd. Redis Documentation[EB/OL]. [2026-05-24]. <https://redis.io/docs/latest>.
- [16] Docker Inc. Docker Documentation[EB/OL]. [2026-05-24]. <https://docs.docker.com>.
- [17] Fette I, Melnikov A. The WebSocket Protocol[S]. RFC 6455, 2011.
- [18] Jones M B, Bradley J, Sakimura N. JSON Web Token (JWT)[S]. RFC 7519, 2015.
- [19] Wang D, Ueda Y, Kula R G, Ishio T, Matsumoto K. Can we benchmark Code Review studies? A systematic mapping study of methodology, dataset, and metric[J]. Journal of Systems and Software, 2021, 180: 111009.
- [20] Alfadel M, Nagy N A, Costa D E, Abdalkareem R, Shihab E. Empirical analysis of security-related code reviews in npm packages[J]. Journal of Systems and Software, 2023, 203: 111752.
- [21] Sharma T, Kechagia M, Georgiou S, Tiwari R, Vats I, Moazen H, Sarro F. A survey on machine learning techniques applied to source code[J]. Journal of Systems and Software, 2024, 209: 111934.
- [22] Sülün E, Saçakcı M, Tüzün E. An empirical analysis of issue templates usage in large-scale projects on GitHub[J]. ACM Transactions on Software Engineering and Methodology, 2024, 33(5): 1-28.

附录

附录 A 主要接口清单

表 A.1 主要业务接口

模块	接口示例	说明
认证注册	POST /auth/register	创建平台用户
认证登录	POST /auth/login	返回用户信息和访问令牌
仓库导入	POST /api/repositories/import	创建仓库与快照任务
仓库详情	GET /api/repositories/:id	查询仓库、分支和互动信息
快照文件树	GET /api/snapshots/:id/tree	读取快照目录结构
快照文件	GET /api/snapshots/:id/file	读取指定文件内容
社区列表	GET /api/community/feed	查询公开仓库 feed
评论创建	POST /api/comments	创建快照、文件或行级评论
通知列表	GET /api/notifications	查询通知并支持标记已读
私信发送	POST /api/chats/:id/messages	发送私信消息
控制台	GET /api/admin/console/overview	查询平台概览和服务状态

附录 B 本地运行命令

```
pnpm install
pnpm db:generate
pnpm -C libs/shared build
pnpm docker:dev
pnpm db:setup
pnpm dev
cd apps/worker && go run .
```

致 谢

本毕业设计从选题、需求分析、系统设计到代码实现和论文撰写，得到了指导教师张坤老师的悉心指导。老师在课题方向、系统边界、功能取舍和论文规范等方面给予了耐心帮助，使我能够将原本较分散的想法逐步收束为一个以代码仓库为核心的开发者社交平台。在此向张坤老师表示衷心感谢。

同时感谢学院各位老师在本科阶段课程学习和实践训练中给予的指导。软件工程、数据库、Web 开发和人工智能相关课程为本课题的完成奠定了基础。感谢同学们在项目调试、功能体验和答辩准备过程中提出的建议。最后感谢家人在学习和生活上的支持，使我能够顺利完成本次毕业设计工作。